

Academic Platform — Scale Analysis (10k Users)

Target load: 10,000 users, 10–50 RPS sustained, bursts to 500+.

This document enumerates failure modes ranked by likelihood at 10k-user scale, with specific file references and mitigations.

Tier 1 — Will break first (days, not months)

1. Postgres connection pool exhaustion

`src/db/index.ts` uses the `postgres` driver with the default `max: 10` connections. Server actions hold a connection for the entire call duration. At 50 concurrent submissions, requests queue, then time out (504).

If deployed to serverless (Vercel, Cloudflare Workers), every cold start opens a fresh pool, causing instant exhaustion.

Fix:

- Put PgBouncer in front of Postgres in transaction-pooling mode.
- Bump `max` to 50 per node, then scale horizontally.
- For serverless, use a pooler-aware connection string (Supabase pooler, Neon pooler, etc.).

2. Piston single-container bottleneck

Compile-heavy languages (Java, C#) consume 1–2 seconds per submission. At 10k users with even one submission per minute, the system sees ~167 RPS to Piston. A single Docker Piston instance dies at roughly 20 concurrent executions. The recent bump to a 10-second timeout for Java/C# makes the queue worse, not better.

Fix:

- Cluster Piston behind a load balancer (3+ instances minimum).
- Or migrate to a managed sandbox service (CodeSandbox API, isolate-on-k8s).

3. AI generation cost spike

`generateMcqs` and the interview LLM calls have a per-user rate limit (`src/modules/assessment/actions.ts:61`) but no global budget cap. If 1,000 admins click "Generate with AI" within the same hour, the platform incurs a four-digit invoice and is hit by provider 429 errors.

Fix:

- Global budget meter (daily \$ ceiling).
- Background queue with backpressure.
- Per-organization spend caps.

4. Submission table unbounded growth

`coding_submissions` , `test_attempts` , and `web_submissions` accumulate without bound.

10,000 users × 5 attempts/day × 365 days = **18 million rows/year per table**.

The `select count(*)` query in `src/modules/coding/service.ts:184-193` runs on every successful submission to detect first-solve. Indexes help, but won't save sequential bloat at this scale.

Fix:

- Partition tables by month (Postgres native partitioning).
- Archive submissions older than 90 days to cold storage (S3/R2).
- Replace count-on-write with a denormalized `first_solved_at` flag on `progress` .

Tier 2 — Degrades within months

5. Monaco editor cold load

The Monaco editor ships ~2–3 MB of JavaScript per first page load. No CDN configuration beyond Next.js defaults. With 10k students hitting first-visit simultaneously (start of class, exam day), the origin server saturates.

Fix:

- Serve `_next/static` from Cloudflare or CloudFront.
- Set immutable cache headers on hashed asset filenames.

6. `getTopicChain` performs a 6-table join on every topic view

Topic, coding, and web pages all call this function. There is no caching layer. Every page load executes the full join.

Fix:

- In-memory LRU cache keyed by `topicId` (5-minute TTL).
- Or denormalize `chapter_name`, `subject_name`, `grade_name` into `topics`.

7. No edge caching for student content

`getTopicContent` and `listGradeTree` re-query Postgres on every request. `force-dynamic` on most pages disables Next.js page-level caching entirely.

Fix:

- Wrap reads in `unstable_cache` with tag-based invalidation.
- Invalidate only on admin edit, not on every read.

8. R2 asset serving proxied through `/api/assets/[id]`

Every PDF and image embed routes through the Next.js runtime instead of redirecting to a signed R2 URL. 10k students viewing the same PDF = 10k Node `fetch` calls, plus 10k transfers through the app server.

Fix:

- Return a 302 redirect to a signed R2 URL.
- Let R2/Cloudflare handle the bytes.

9. `bcryptjs` (pure JS) password hashing

`package.json` uses `bcryptjs`, which is roughly 10× slower than native `bcrypt`. A login burst (start of class = 500 logins/minute) burns CPU.

Fix:

- Swap to native `bcrypt` bindings.
- Or migrate to `argon2` (modern, faster, harder to brute force).

Tier 3 — Long tail

10. NextAuth session strategy

If sessions are stored in the database, every request incurs an extra query. Confirm the project is using JWT-mode sessions. If JWT, this is a non-issue.

11. `revalidatePath` cascade

Admin edits trigger page cache invalidation. Many admins editing simultaneously cause cache thrash. Manageable at 10k users, but worth monitoring.

12. MCQ bulk import is in-memory

A 5 MB upload × 100 concurrent uploads = 500 MB Node heap spike. Single instance OOM is possible.

Fix:

- Stream-parse the file.
- Cap to one concurrent import per admin.

13. Web-mode iframe checks

These scale for free — client-side compute. No failure here.

14. Progress upserts

Lock the `(student_id, topic_id)` row. Fine at scale.

15. No queue for submissions

Submissions run synchronously inside the server action. If Piston is slow, the entire request times out and the user sees an error.

Fix:

- Enqueue the submission, return a job ID.
- Poll or push results via SSE / WebSocket.

What is already solid

- Drizzle query shapes — no N+1 except topic chain.

- Auth guards on every server action.
- Indexes on hot paths (`student_idx` , `topic_idx` , `q_idx`).
- R2 for binaries (not stored as DB blobs).
- Iframe sandbox isolation for web mode.

Minimum work to survive 10k users

Priority	Item	Effort
P0	PgBouncer + bump pool to 50	0.5 day
P0	Piston ×3 behind LB	1 day
P0	Move static assets to CDN	0.5 day
P0	<code>unstable_cache</code> on hot reads	1 day
P0	302 redirect for R2 assets	0.5 day
P0	Native bcrypt	0.5 day
P0	Global AI budget cap	0.5 day
P1	Partition submission tables	1 day
P1	Submission queue + SSE	2 days

Total: ~7 days engineering for Tier 1 + Tier 2 mitigations.

Without these, expect cascading failure at approximately **500 concurrent users** — well below the 10k target.